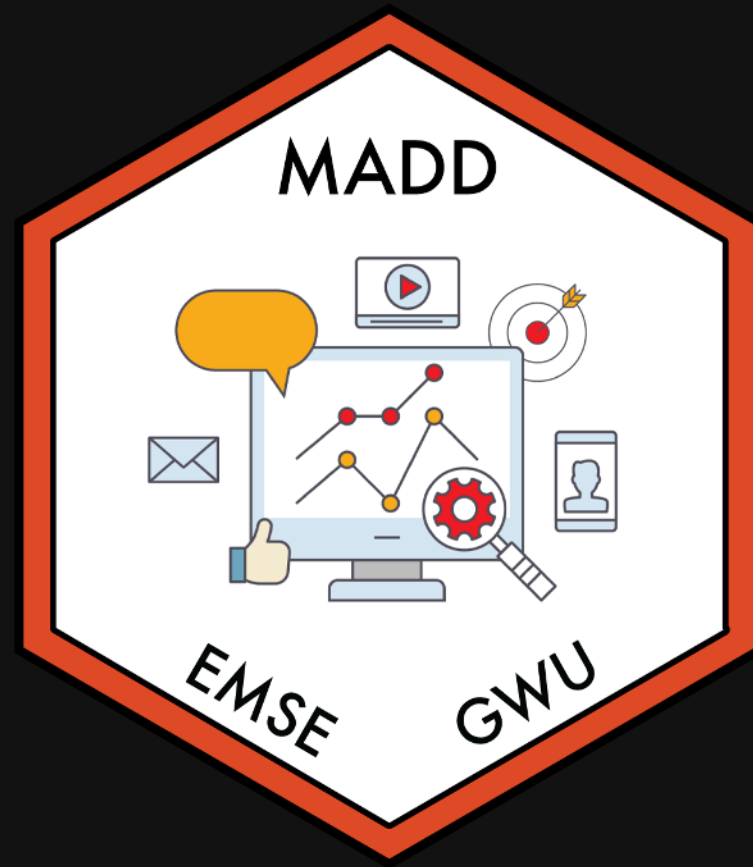




Conjoint Questions

🏛️ EMSE 6035: Marketing Analytics for Design Decisions

👤 John Paul Helveston

📅 October 07, 2026

Some Quarto tips

Convert a data frame to a markdown table with `kable()`

```
1 library(tidyverse)
2
3 mtcars %>%
4   kable()
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

Example from last year

```
1 library(tidyverse)
2
3 df <- read_csv("competitors.csv")
4 df %>%
5   kable()
```

brand	volumelnOz	quantity	price	pricePerCup	biodegradability	opacity	logo
greatValue	9	50	\$2.98	\$0.06	FALSE	TRUE	FALSE
decorRack	9	50	\$5.99	\$0.12	FALSE	TRUE	FALSE
tigerChef	9	100	\$7.99	\$0.08	FALSE	TRUE	FALSE
smartly	9	80	\$2.79	\$0.03	FALSE	FALSE	FALSE
solo	9	50	\$4.04	\$0.08	FALSE	FALSE	FALSE
greatValue	9	100	\$3.76	\$0.04	FALSE	FALSE	FALSE
ecoProducts	9	1000	\$187.69	\$0.19	TRUE	FALSE	TRUE
worldCentric	9	2000	\$220.00	\$0.11	TRUE	FALSE	TRUE
hefty	18	50	\$3.98	\$0.08	FALSE	TRUE	FALSE

More `kable()` formatting options:
`{kableExtra}` package

References

Simple approach: Insert a footnote with `^[]`

This markdown...

```
The Eiffel Tower is 324 meters  
tall^[From the [Eiffel Tower wikipedia  
page](https://en.wikipedia.org/wiki  
/Eiffel_Tower)]
```

...renders as this

The Eiffel Tower is 324 meters tall¹

¹From the [Eiffel Tower wikipedia page](https://en.wikipedia.org/wiki/Eiffel_Tower)

References

Complex (but more complete) approach: Use bibtex

<https://quarto.org/docs/authoring/footnotes-and-citations.html>

You can insert citations with `[@citekey]`, and a “References” table will be automatically created.

Footnotes are perfectly fine for this class

Week 7: *Conjoint Questions*

1. Defining choice questions in R
2. Displaying choice questions in surveydown

Week 7: *Conjoint Questions*

1. Defining choice questions in R
2. Displaying choice questions in surveydown

1. Download today's notes, then
2. Open the `conjoint_buttons` folder in Positron

We'll be using the `{cbcTools}` package today

Install with:

```
1 remotes::install_github("jhelvy/cbcTools")
```

Choice question components

1. Generate **profiles** for each attribute and level
2. Create a survey **design** data frame from **profiles**

Basic Design

Any combination of attributes can be shown in each choice question

Question 1

Option:	1	2	3
Type:	Fuji	Pink Lady	Fuji
Price:	\$4.00	\$1.50	\$2.50
Freshness:	Excellent	Poor	Average

Question 2

Option:	1	2	3
Type:	Pink Lady	Gala	Gala
Price:	\$2.00	\$4.00	\$1.50
Freshness:	Excellent	Average	Average

This is exactly what's in

code/1_make_choice_questions.R

```
1 profiles <- cbc_profiles(  
2   type = c("Fuji", "Gala", "Honeycrisp", "Pink Lady", "Red Delicious"),  
3   price = seq(1, 4, 0.5), # Price ($ / lb)  
4   freshness = c("Excellent", "Average", "Poor")  
5 )  
6  
7 design <- cbc_design(  
8   profiles = profiles,  
9   n_resp = 2000, # Number of respondents  
10  n_alts = 3, # Number of alternatives per question  
11  n_q = 6 # Number of questions per respondent  
12 )
```

Your Turn

15:00

Open the `conjoint_buttons` template folder in Positron

In `code/1_make_choice_questions.R`, replace the apple `profiles` with your own team's attributes and levels

Leave `cbc_design()` at its default values for now - just get a basic design of your own running

Three design choices to consider

Three design choices to consider

1. Regular or labeled?
2. Include a “none” option (outside good) or not?
3. Include restrictions?

1. Regular or labeled?

A **labeled** design fixes one attribute to a specific option position
e.g. Option 1 is always "Local", option 2 is always "Imported"

Question 1

Option:	Local	Imported
Type:	Fuji	Honeycrisp
Price:	\$2.50	\$2.00
Miles traveled:	500 mi	1500 mi

Question 2

Option:	Local	Imported
Type:	Gala	Fuji
Price:	\$2.00	\$1.50
Miles traveled:	500 mi	3000 mi

Code: labeled design

```
1 design_labeled <- cbc_design(  
2   profiles = profiles,  
3   n_resp = 500,  
4   n_alts = 2, # must equal the # of levels in the label attribute  
5   n_q = 6,  
6   label = "sourcing"  
7 )
```

Use a labeled design when respondents are really comparing **named alternatives** (brands, powertrains, sourcing...) rather than unlabeled bundles of attributes

2. Include a “none” option?

A “none” option lets respondents say “I wouldn’t buy any of these”

Question 1

Option:	1	2	3	None
Type:	Honeycrisp	Gala	Fuji	
Price:	\$4.00	\$2.00	\$1.00	
Freshness:	Average	Average	Poor	

Question 2

Option:	1	2	3	None
Type:	Pink Lady	Honeycrisp	Gala	
Price:	\$1.50	\$3.50	\$3.50	
Freshness:	Poor	Poor	Excellent	

Code: a “none” option

```
1 design_no_choice <- cbc_design(  
2   profiles = profiles,  
3   n_resp = 2000,  
4   n_alts = 3,  
5   n_q = 6,  
6   no_choice = TRUE  
7 )
```

Without this, respondents are forced to pick one of the alternatives shown, even if they wouldn't choose any of them in real life

Attribute-specific features

Some attributes may only be valid for certain levels of other attributes

Example: “Miles traveled” only makes sense for **imported** apples (locally grown apples don’t really have a meaningful travel distance).

To implement this, add restrictions to your `profiles` with `cbc_restrict()`

Code: restrictions

```
1 profiles <- cbc_profiles(  
2   type = c("Fuji", "Gala", "Honeycrisp"),  
3   price = seq(1, 4, 0.5),  
4   sourcing = c("Local", "Imported"),  
5   milesTraveled = c(50, 500, 1500, 3000)  
6 )  
7  
8 # Local apples only get the lowest ("~0") miles traveled value  
9 profiles_restricted <- profiles %>%  
10   cbc_restrict(  
11     sourcing == "Local" & milesTraveled > 50  
12   )
```

Warning: Restrictions can lead to imbalances

Design the questions from the restricted profiles...

```
1 design_restricted <- cbc_design(  
2   profiles = profiles_restricted,  
3   n_resp = 500,  
4   n_alts = 2,  
5   n_q = 6  
6 )
```

```
#>  
#>      Local Imported  
#>      1171      4829
```

"Imported" apples show up far more often, since they have more valid attribute combinations left after restricting

Fix imbalances with `balance_by`

```
1 design_balanced <- cbc_design(  
2   profiles = profiles_restricted,  
3   n_resp = 500,  
4   n_alts = 2,  
5   n_q = 6,  
6   balance_by = "sourcing"  
7 )
```

```
#>  
#>   Local Imported  
#>   3011    2989
```

Now "Local" and "Imported" show up about equally often

Your Turn

20:00

With your team, discuss the specific choice question design for your project

- Regular or labeled?
- Include a "none" option (outside good) or not?
- Include restrictions?

Then update `code/1_make_choice_questions.R` to match

Week 7: *Conjoint Questions*

1. Defining choice questions in R
2. Displaying choice questions in surveydown

Displaying choice questions in your survey

1. Read in the design file (`choice_questions.csv`)
2. Sample and store a random respondent ID
3. Filter the design for the respondentID
4. Define a function to create question options
5. Create the options for each choice question
6. Create each choice question (6 in total)

1. Read in the design file (`choice_questions.csv`)

```
1 design <- readr::read_csv("choice_questions.csv")
```

2. Sample and store a random respondent ID

Sample a random respondentID

```
1 respondentID <- sample(design$respID, 1)
```

Store the respondentID

```
1 sd_store_value(respondentID, "respID")
```

In your survey data, you will have a `respID` column

3. Filter the design for the respondentID

```
1 df <- design %>%
2   filter(respID == respondentID) %>%
3   mutate(image = paste0("images/", image))
```

Images are stored in "images" folder, so we paste on that path

Result looks like this:

```
1 head(df)
```

```
#> # A tibble: 6 × 9
#>   profileID respID   qID altID obsID type      price freshness image
#>   <dbl>   <dbl> <dbl> <dbl> <dbl> <chr>    <dbl> <chr>    <chr>
#> 1      45     28     1     1   163 Red Delicious  1.5 Average images/redDe
#> 2      77     28     1     2   163 Gala        1.5 Poor    images/gala
#> 3      94     28     1     3   163 Pink Lady    3    Poor    images/pinkl
#> 4       8     28     2     1   164 Honeycrisp  1.5 Excellent images/honey
#> 5      85     28     2     2   164 Red Delicious  2    Poor    images/redDe
#> 6      80     28     2     3   164 Red Delicious  1.5 Poor    images/redDe
```

4. Define a function to create question options

```
1 make_cbc_options <- function(df) {  
2   alt1 <- df |> filter(altID == 1)  
3   alt2 <- df |> filter(altID == 2)  
4   alt3 <- df |> filter(altID == 3)  
5  
6   options <- c("option_1", "option_2", "option_3")  
7  
8   names(options) <- c(  
9     glue(  
10      "  
11      **Option 1**<br>  
12      <img src='{alt1$image}' width=100><br>  
13      **Type**: {alt1$type}<br>  
14      **Price**: $ {alt1$price} / lb  
15      "  
16    ),  
17    glue(  
18      "  
19      **Option 2**<br>  
20      <img src='{alt2$image}' width=100><br>  
21      **Type**: {alt2$type}<br>  
22      **Price**: $ {alt2$price} / lb  
23      "  
24    ),  
25    glue(  
26      "  
27      **Option 3**<br>  
28      <img src='{alt3$image}' width=100><br>  
29      **Type**: {alt3$type}<br>  
30      **Price**: $ {alt3$price} / lb  
31      "  
32    )  
33  )  
34  return(options)  
35 }
```

4. Define a function to create question options

Make one-row data frames for each alternative.

For example, `alt1` looks like this:

```
#> Rows: 1
#> Columns: 9
#> $ profileID <dbl> 45
#> $ respID    <dbl> 28
#> $ qID       <dbl> 1
#> $ altID     <dbl> 1
#> $ obsID     <dbl> 163
#> $ type      <chr> "Red Delicious"
#> $ price     <dbl> 1.5
#> $ freshness <chr> "Average"
#> $ image     <chr> "images/redDelicious.jpg"
```

```
1 make_cbc_options <- function(df) {
2   alt1 <- df |> filter(altID == 1)
3   alt2 <- df |> filter(altID == 2)
4   alt3 <- df |> filter(altID == 3)
5
6   options <- c("option_1", "option_2", "option_3")
7
8   names(options) <- c(
9     glue(
10      "
11      **Option 1**<br>
12      <img src='{alt1$image}' width=100><br>
13      **Type**: {alt1$type}<br>
14      **Price**: $ {alt1$price} / lb
15      "
16    ),
17    glue(
18      "
19      **Option 2**<br>
20      <img src='{alt2$image}' width=100><br>
21      **Type**: {alt2$type}<br>
22      **Price**: $ {alt2$price} / lb
23      "
24    ),
25    glue(
26      "
27      **Option 3**<br>
28      <img src='{alt3$image}' width=100><br>
29      **Type**: {alt3$type}<br>
30      **Price**: $ {alt3$price} / lb
31      "
32    )
33  )
34  return(options)
35 }
```

4. Define a function to create question options

```
1 alt1[c('image', 'type', 'price')]
```

```
#> # A tibble: 1 × 3  
#>   image          type      price  
#>   <chr>        <chr>    <dbl>  
#> 1 images/redDelicious.jpg Red Delicious  1.5
```

Renders as:

```
#> **Option 1**<br>  
#> <img src='images/redDelicious.jpg' width=100><br>  
#> **Type**: Red Delicious<br>  
#> **Price**: $ 1.5 / lb
```

```
1 make_cbc_options <- function(df) {  
2   alt1 <- df |> filter(altID == 1)  
3   alt2 <- df |> filter(altID == 2)  
4   alt3 <- df |> filter(altID == 3)  
5  
6   options <- c("option_1", "option_2", "option_3")  
7  
8   names(options) <- c(  
9     glue(  
10      "  
11      **Option 1**<br>  
12      <img src='{alt1$image}' width=100><br>  
13      **Type**: {alt1$type}<br>  
14      **Price**: $ {alt1$price} / lb  
15      "  
16    ),  
17    glue(  
18      "  
19      **Option 2**<br>  
20      <img src='{alt2$image}' width=100><br>  
21      **Type**: {alt2$type}<br>  
22      **Price**: $ {alt2$price} / lb  
23      "  
24    ),  
25    glue(  
26      "  
27      **Option 3**<br>  
28      <img src='{alt3$image}' width=100><br>  
29      **Type**: {alt3$type}<br>  
30      **Price**: $ {alt3$price} / lb  
31      "  
32    )  
33  )  
34  return(options)  
35 }
```

5. Create the options for each choice question

```
1 cbc1_options <- make_cbc_options(df |> filter(qID == 1))
```

Value stored in your data:

```
#> option_1
```

Label:

```
#> [1] " **Option 1**<br>\n  <img src=
```

Renders as:

****Option 1****



****Type****: Red Delicious

****Price****: \$ 1.5 / lb

5. Create the options for each choice question

Now make the options for each question

```
1 cbc1_options <- make_cbc_options(df |> filter(qID == 1))
2 cbc2_options <- make_cbc_options(df |> filter(qID == 2))
3 cbc3_options <- make_cbc_options(df |> filter(qID == 3))
4 cbc4_options <- make_cbc_options(df |> filter(qID == 4))
5 cbc5_options <- make_cbc_options(df |> filter(qID == 5))
6 cbc6_options <- make_cbc_options(df |> filter(qID == 6))
```

6. Create each choice question (6 in total)

Code:

```
1 sd_question(  
2     type = 'mc_buttons',  
3     id = 'cbc_q1',  
4     label = "(1 of 6) If these were your only choices, which would you choose?",  
5     option = cbc1_options  
6 )
```

Renders as:

Option 1	Option 2	Option 3
		
Type: Red Delicious Price: \$ 1 / lb	Type: Red Delicious Price: \$ 2 / lb	Type: Fuji Price: \$ 2 / lb

TABLE layout: Show options in a table with `kable()`

```
1 library(dplyr)
2
3 alts <- df %>%
4   filter(qID == 1) %>%
5   mutate(
6     # Add $ sign to price and html code for image path
7     price = paste(scales::dollar(price), "/ lb"),
8     image = paste0('<img src="", image, '" width=100>')
9   ) %>%
10  # Make nicer attribute labels
11  select(
12    `Option:` = altID,
13    ` ` = image,
14    `Price:` = price,
15    `Type:` = type,
16    `Freshness:` = freshness
17  )
18
19 # Drop row names
20 row.names(alts) <- NULL
```

Option: Show options in a table with `kable()`

Display the *transpose*, `t(alts)`

```
1 kbl(t(alts), escape = FALSE) %>%  
2   kable_styling(  
3     bootstrap_options = c("striped", "hover", "condensed"),  
4     full_width = FALSE,  
5     position = "center"  
6   )
```

Option:	1	2	3
			
Price:	\$1.50 / lb	\$1.50 / lb	\$3.00 / lb
Type:	Red Delicious	Gala	Pink Lady
Freshness:	Average	Poor	Poor

TABLE layout: Show options in a table with `kable()`

Your choice question can be defined in the `survey.qmd` file as it is just choosing one of three options:

If these were your only options, which would you choose?

Option 1 Option 2 Option 3

Option:	1	2	3
			
Price:	\$4.00 / lb	\$2.50 / lb	\$4.00 / lb
Type:	Fuji	Gala	Honeycrisp
Freshness:	Excellent	Excellent	Average

Next

Your Turn

20:00

With your team, choose a format to work with (buttons or table) for your choice question, then modify the example survey to match your project.

Versions:

Buttons layout: `conjoint_buttons`

Table layout: `conjoint_tables`